الأكاديمية العربية للعلوم والتكنولوجيا والنقل البحري
Arab Academy for Science, Technology & Maritime Transport
College of Computing and Information Technology

CS460/CAI3105

DEEP LEARNING

LECTURE-1

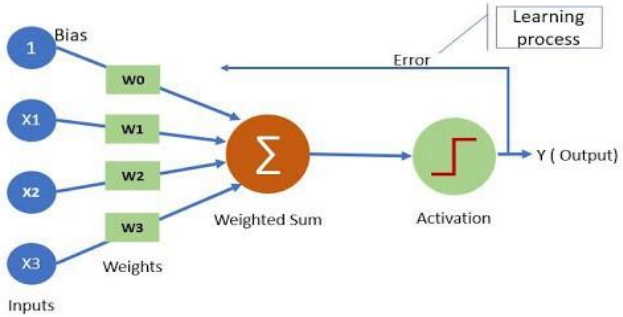
Artificial Neural Networks (ANN)
Introduction - Single Layer Perceptron

Introduction

- An Artificial Neural Network (ANN) is either a hardware implementation or a computer program, which strives to simulate the information processing capabilities of its biological exemplar.
- ANNs are typically composed of a great number of interconnected artificial neurons. The artificial neurons are simplified models of their biological counterparts.
- ANN is a technique for solving problems by constructing software that works like our brains.

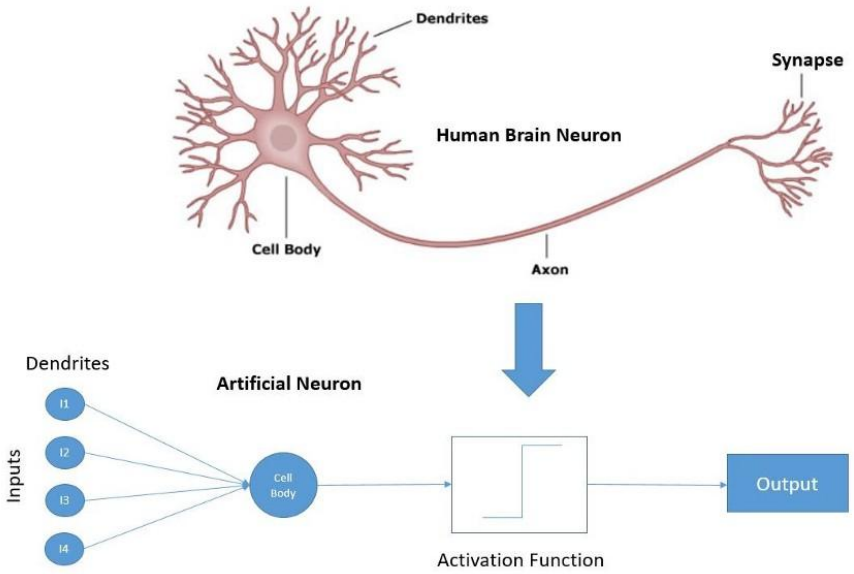
Artificial Neural Network (ANN)

- An Artificial Neural Network (ANN) is a computational model inspired by networks of biological neurons, wherein the neurons compute output values from inputs.
- ANNs are simplified models of the biological nervous system, and therefore have been motivated by the kind of computing performed by the human brain.
- The behavior of a biological neural network can be captured by a simple model called artificial neuron or (**Perceptron**).



Perceptron

Image source: https://laptrinhx.com/gender-prediction-using-a-neural-



ANN learning

- An ANN can:
  1. Compute *any computable* function, by the appropriate selection of the network topology and weights values.
  2. Learn from experience → Continuous process of:

     ➢Training:

     Processing an input to produce an output (compute the output function of a given input).

     ➢Evaluation:

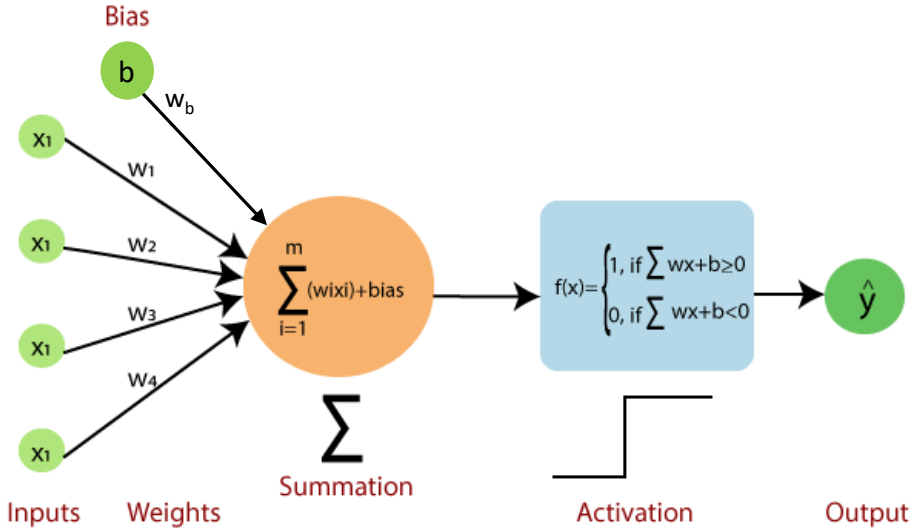
     Evaluating the output by comparing the actual output with the expected output.

     ➢Tuning:

     Adjusting the weights based on the calculated Loss value.

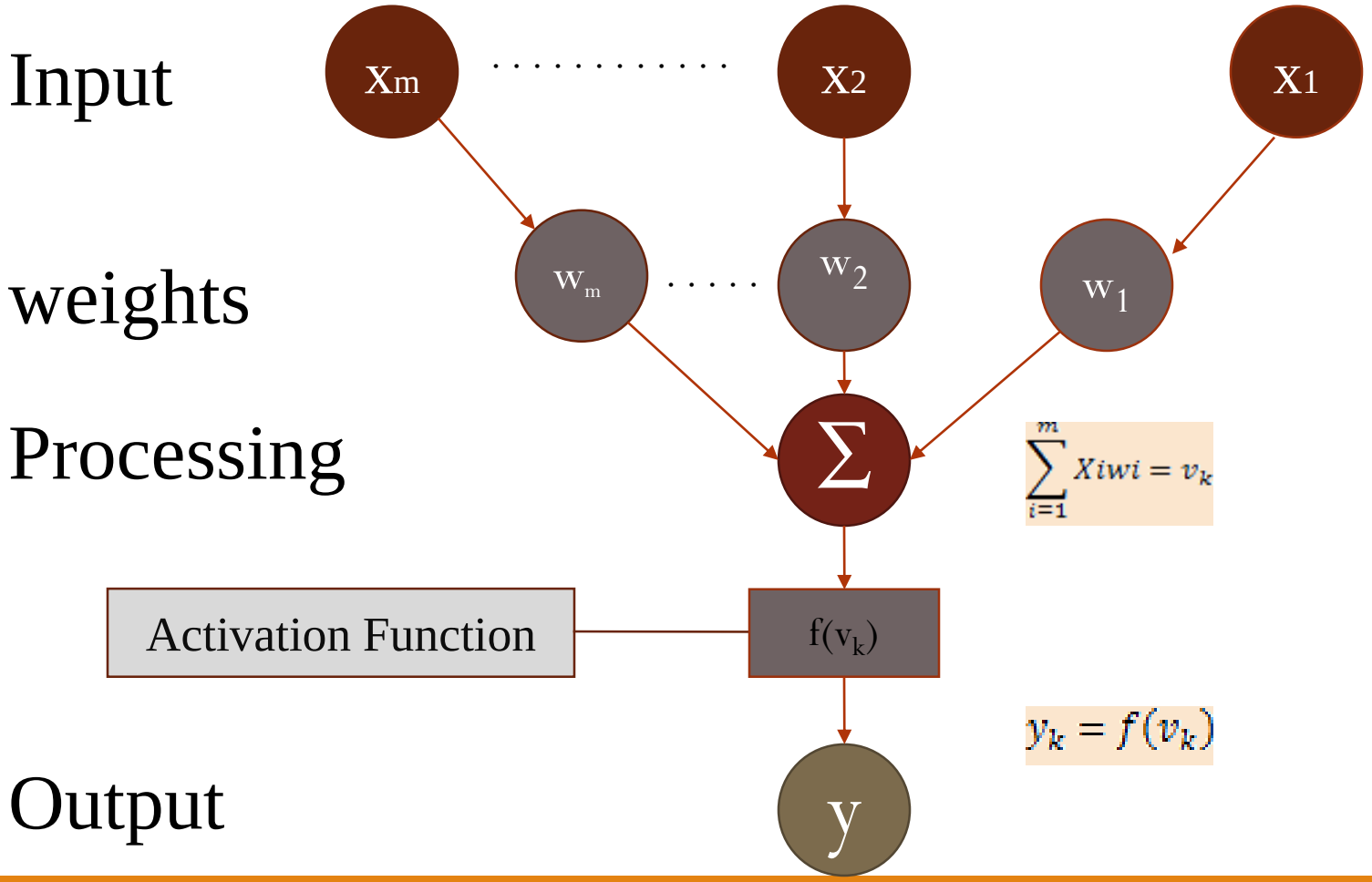
Single-Layer Perceptron (SLP)

- A single-layer artificial neural network, also called a Single-Layer Perceptron (SLP), has a single layer of nodes. Each node in the single layer connects directly to an input variable and contributes to an output variable.
- SLPs have just one layer of active units. Inputs connect directly to the outputs through a single layer of weights. The outputs do not interact, so a network with N outputs can be treated as N separate single-output networks.



Functionality of ANN

- The output is a function of the input, that is affected by the weights, and the activation functions.



Linear separability

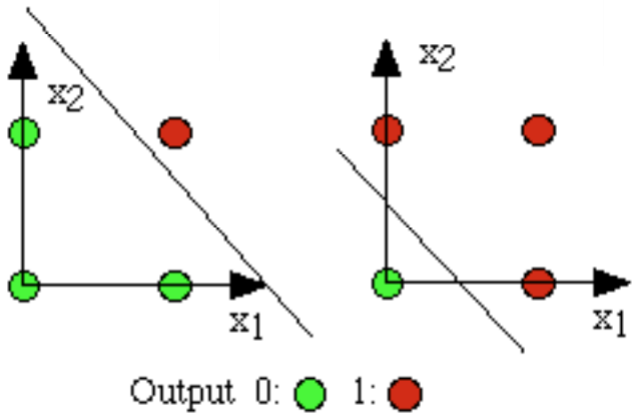
- Two sets are linearly separable if there's a classifier whose decision boundary separates the objects of one set from the objects of the other set (with a straight line in 2D dimension or a hyper plane in higher dimensions).

Consider a single-layer perceptron

- Assume threshold units
- Assume binary inputs and outputs
- Weighted sum forms a linear hyperplane

Consider a single output network with two inputs

- Only functions that are linearly separable can be computed
- Example: AND is linearly separable; OR is linearly separable



AND logic function OR logic function

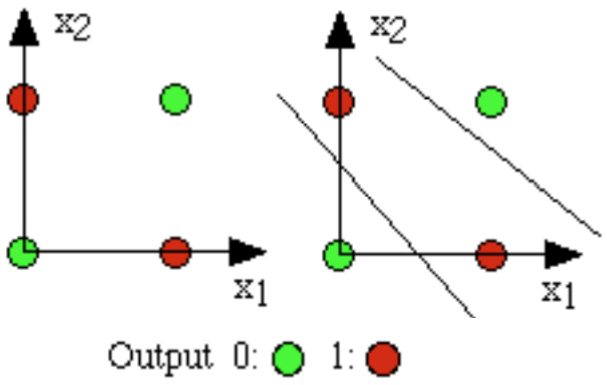
Linear inseparability

- A set of input vectors (or a training set) will be said to be linearly non-separable if no line or hyperplane exists such that each vector lies on the pre-assigned side of that line or hyperplane.

→ A single layer of perceptron cannot be used to solve linearly inseparable problems.

→ This is why the XOR problem cannot be solved by One layer perceptron.

- Single-layer perceptron with threshold units fails if problem is not linearly separable
  - Example: XOR



XOR logic function

Activation function

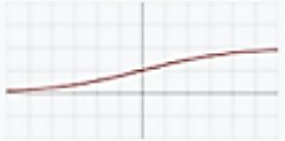
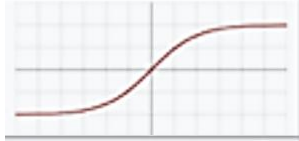
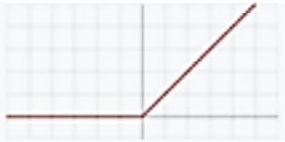
- Activation function of a node in an artificial neural network is a function that calculates the output of the node (based on its inputs and the weights on individual inputs).
- A neural network activation function is **a function that is applied to the output of a neuron**.
- The activation function **decides whether a neuron should be activated or not by calculating the weighted sum and** further adding bias to it.
- Activation functions in ANNs introduce an additional step at each layer during the forward propagation.
- Activation functions in ANNs define how the weighted sum of the input is transformed into an output from a node or nodes in a layer of the network.

→Suppose a neural network working without the activation functions!
→In that case, every neuron will only be performing a linear transformation on the inputs using the weights and biases.

Without using activation functions, although the neural network becomes simpler, learning any complex task is impossible, and our model would be just a linear regression model.

Activation functions

- Binary (Step, threshold-based)
- Sigmoid (Logistic)
- Tanh
- Rectified Linear Unit (ReLU)

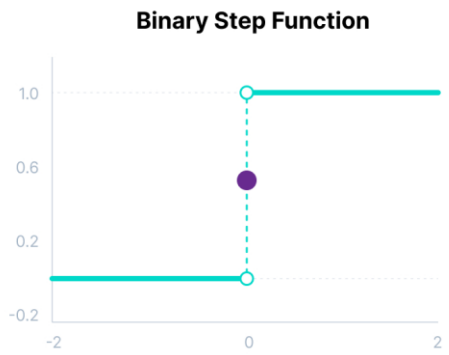
	$f(x) = \begin{cases} 0 & \text{for} \quad x < 0 \\ 1 & \text{for} \quad x \geq 0 \end{cases}$
	$f(x) = \frac{1}{1+e^{-x}}$
	$f(x) = \tanh(x) = \frac{2}{1+e^{-2x}} - 1$
	$f(x) = \begin{cases} 0 & \text{for} \quad x < 0 \\ x & \text{for} \quad x \geq 0 \end{cases}$

Activation functions

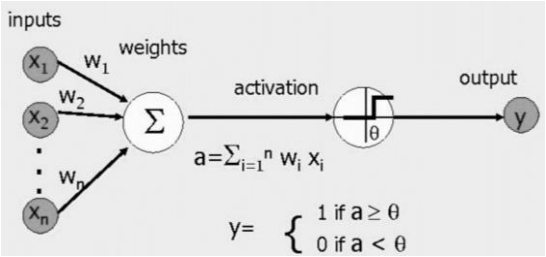
- **Example Linear activation function:**

- **Binary Step Function:** depends on a threshold value that decides whether a neuron should be activated or not. The input fed to the activation function is compared to a certain threshold; if the input is greater than it, then the neuron is activated, else it is deactivated.

$$f(x) = \begin{cases} 0 & for\ x < 0 \\ 1 & for\ x \geqslant 0 \end{cases}$$



→ limitations of binary step function:
- It cannot provide multi-value outputs (it cannot be used for multi-class classification problems).
- The gradient of the step function is zero, which causes difficulty for the backpropagation process.



Activation functions

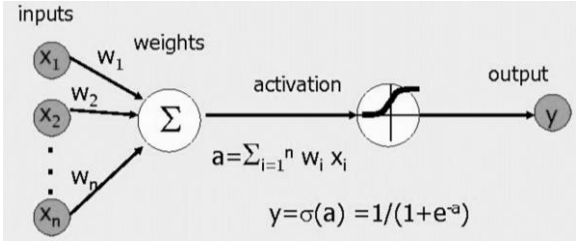
➢ **Example Non-linear activation function:**

→ Non-linear activation functions solve the following limitations of linear activation functions:

- Allow backpropagation as it's possible to go back and understand which weights in the input neurons can provide a better prediction.
- Allow the stacking of multiple layers of neurons (Multi Layer Perceptron – MLP) as the output would be a non-linear combination of input passed through multiple layers.

- **Sigmoid Function:** This function takes any real value as input and outputs values in the range of 0 to 1.

The larger the input (more positive), the closer the output value will be to 1.0, whereas the smaller the input (more negative), the closer the output will be to 0.0

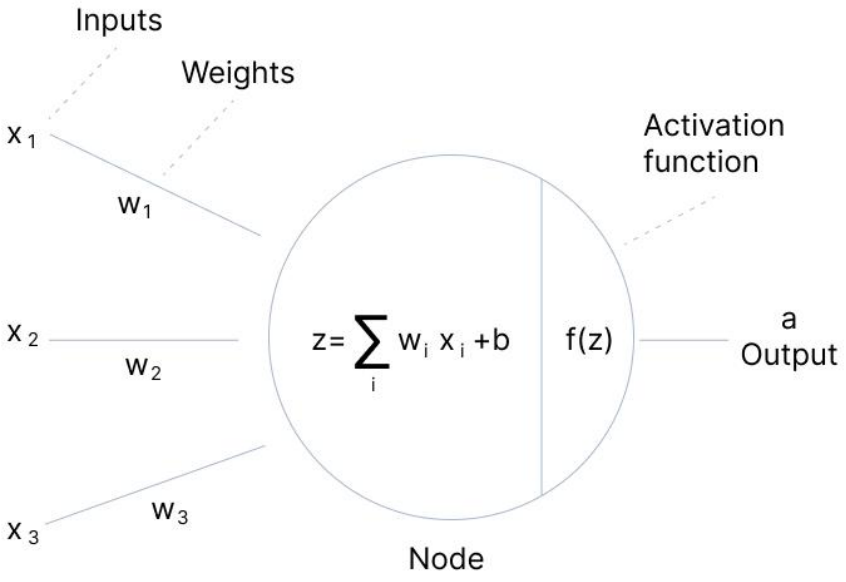


$$f(x) = \frac{1}{1 + e^{-x}}$$

Single Layer Perceptron (SLP)

Single Layer Perceptron (SLP)

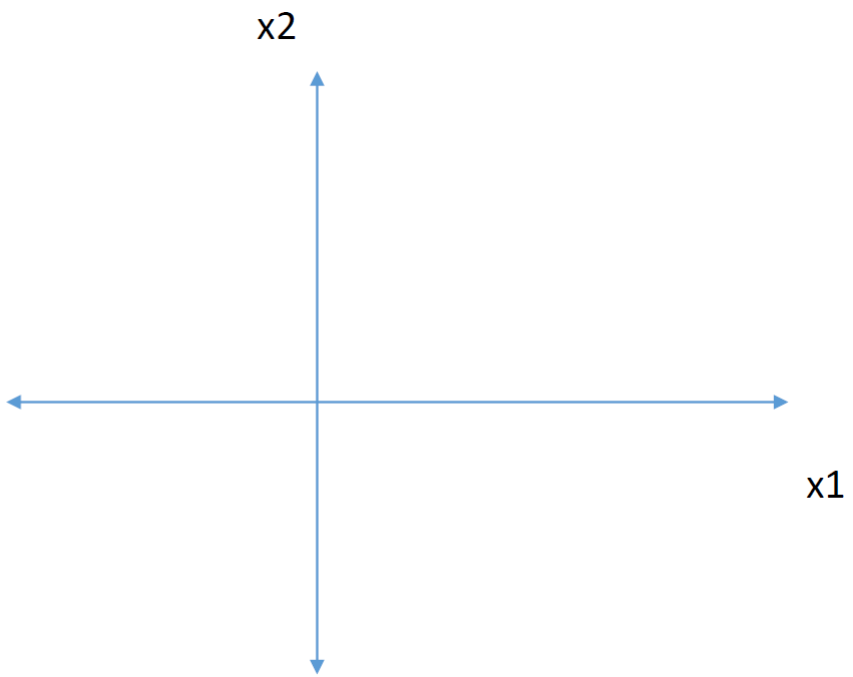
- A single layer perceptron (SLP) is a feed-forward network based on a threshold activation function.
- SLP is the simplest type of ANN and can only classify linearly separable cases with a binary target (1 , 0).



Single Layer Perceptron (SLP)

- Visualization example (2 features):

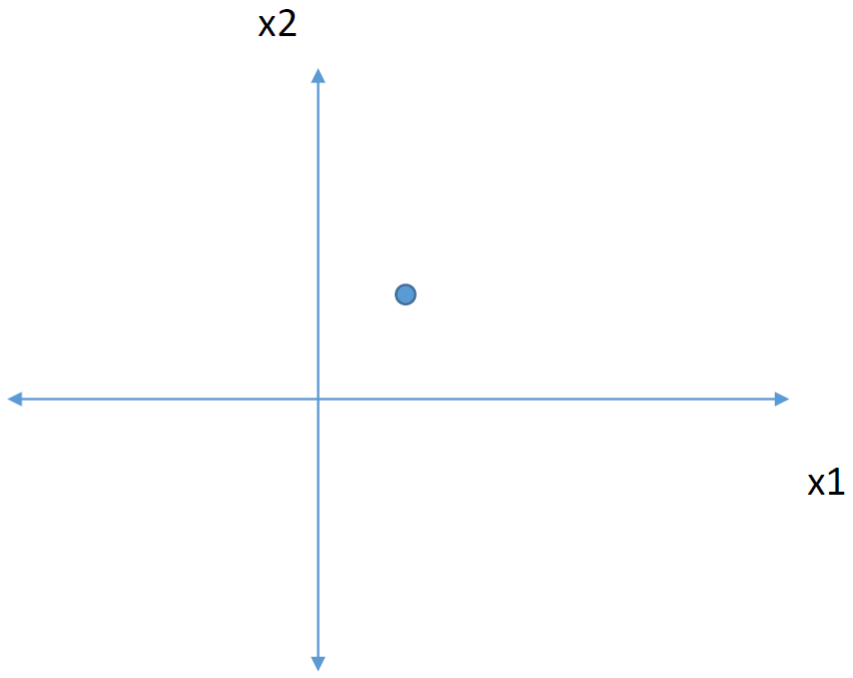
x1	x2	t
2	3	0
-3	3	1
3	4	0
-1	2	1
7	2	0
0	1	1
0	-2	1



Single Layer Perceptron (SLP)

- Visualization example (2 features):

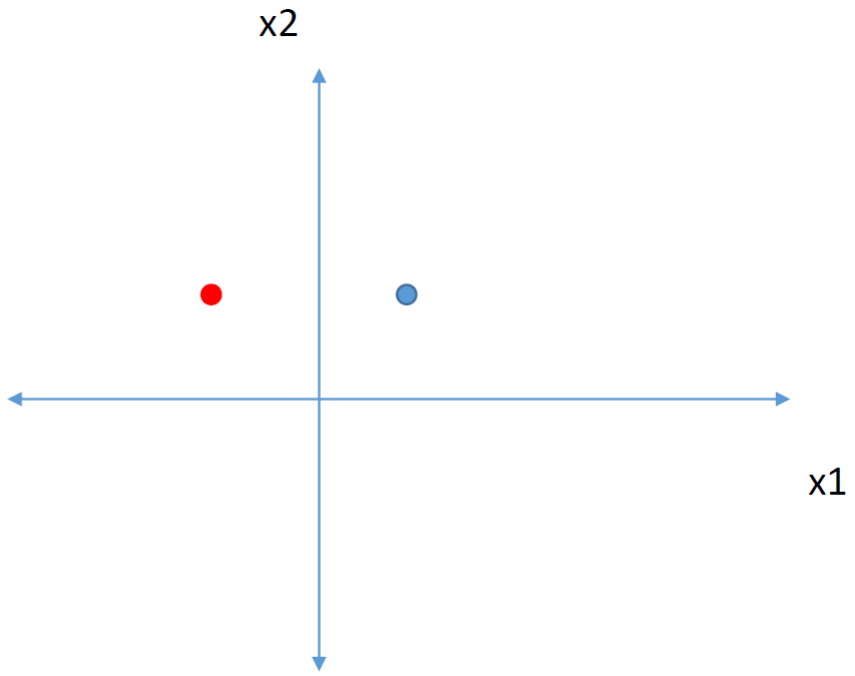
x1	x2	t
2	3	0
-3	3	1
3	4	0
-1	2	1
7	2	0
0	1	1
0	-2	1



Single Layer Perceptron (SLP)

- Visualization example (2 features):

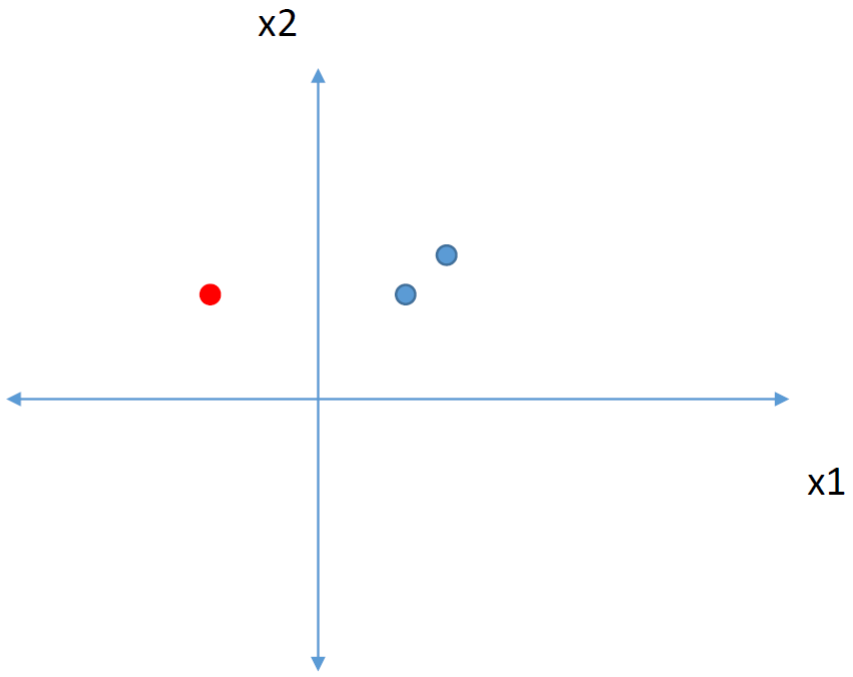
x1	x2	t
2	3	0
-3	3	1
3	4	0
-1	2	1
7	2	0
0	1	1
0	-2	1



Single Layer Perceptron (SLP)

- Visualization example (2 features):

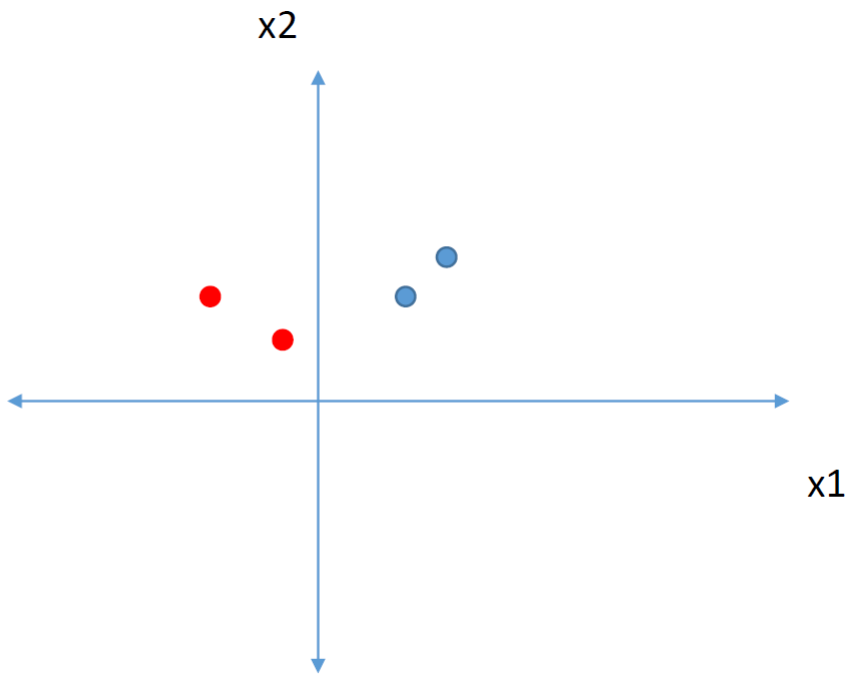
x1	x2	t
2	3	0
-3	3	1
3	4	0
-1	2	1
7	2	0
0	1	1
0	-2	1



Single Layer Perceptron (SLP)

- Visualization example (2 features):

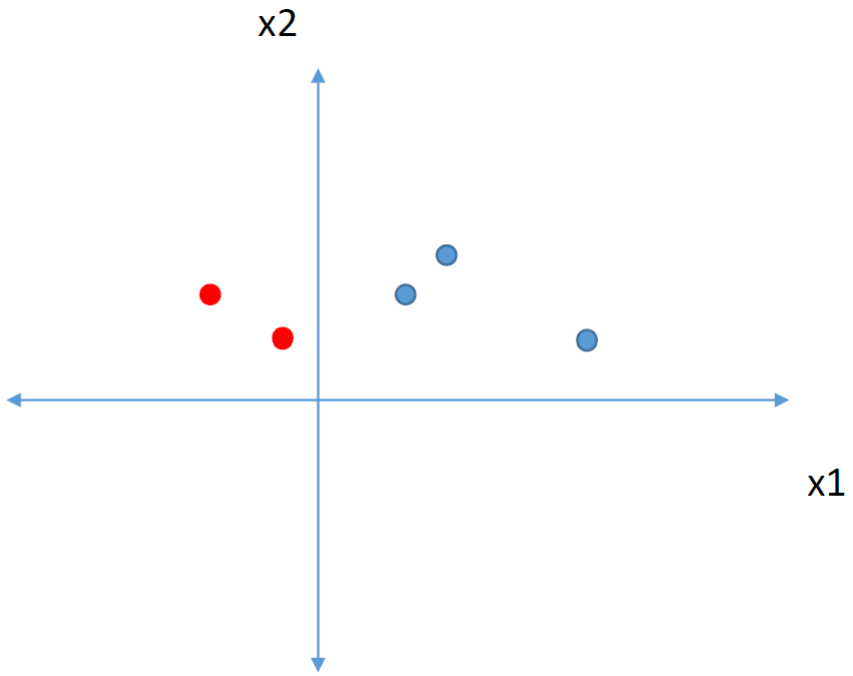
x1	x2	t
2	3	0
-3	3	1
3	4	0
-1	2	1
7	2	0
0	1	1
0	-2	1



Single Layer Perceptron (SLP)

- Visualization example (2 features):

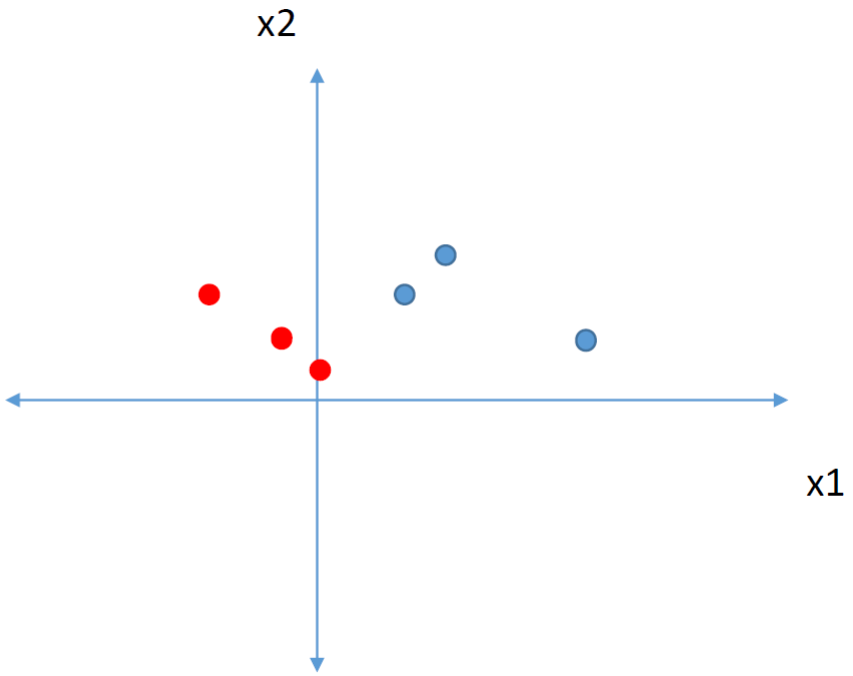
x1	x2	t
2	3	0
-3	3	1
3	4	0
-1	2	1
7	2	0
0	1	1
0	-2	1



Single Layer Perceptron (SLP)

- Visualization example (2 features):

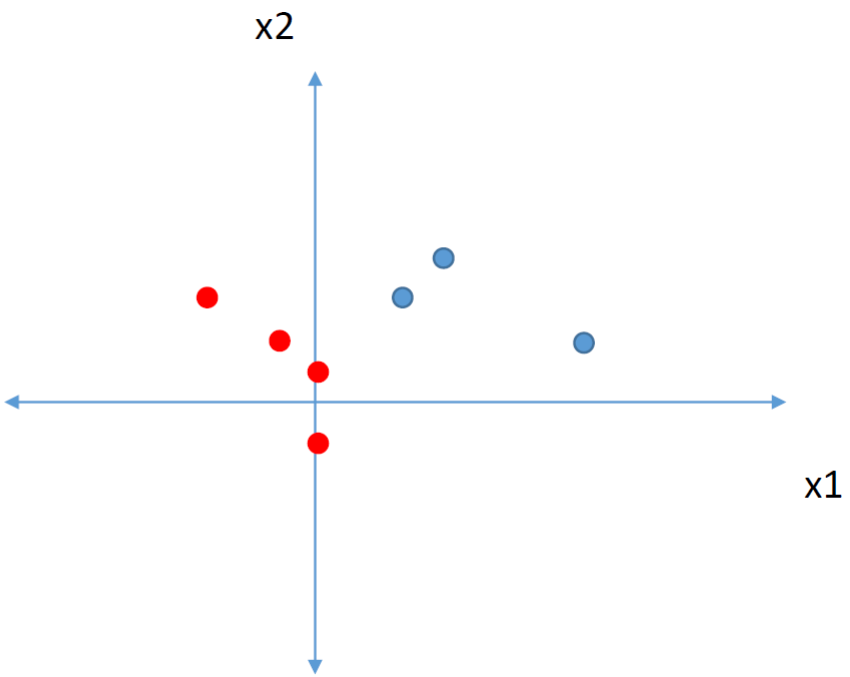
x1	x2	t
2	3	0
-3	3	1
3	4	0
-1	2	1
7	2	0
0	1	1
0	-2	1



Single Layer Perceptron (SLP)

- Visualization example (2 features):

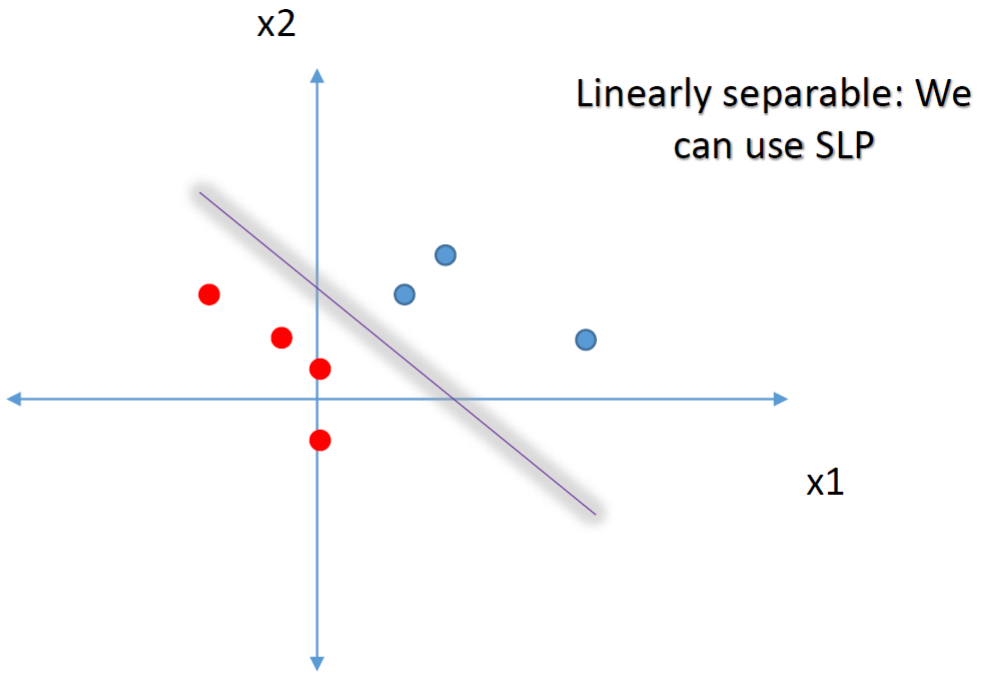
x1	x2	t
2	3	0
-3	3	1
3	4	0
-1	2	1
7	2	0
0	1	1
0	-2	1



Single Layer Perceptron (SLP)

- Visualization example (2 features):

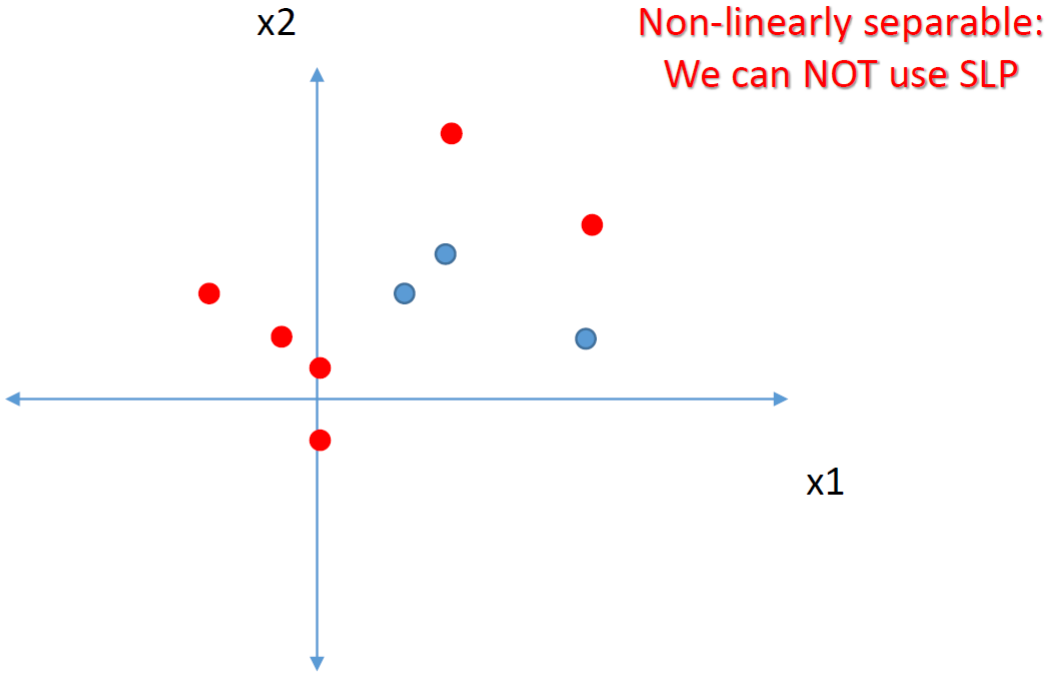
x1	x2	t
2	3	0
-3	3	1
3	4	0
-1	2	1
7	2	0
0	1	1
0	-2	1



Single Layer Perceptron (SLP)

- Visualization example (2 features):

x1	x2	t
2	3	0
-3	3	1
3	4	0
-1	2	1
7	2	0
0	1	1
0	-2	1
3	8	1
7	5	1



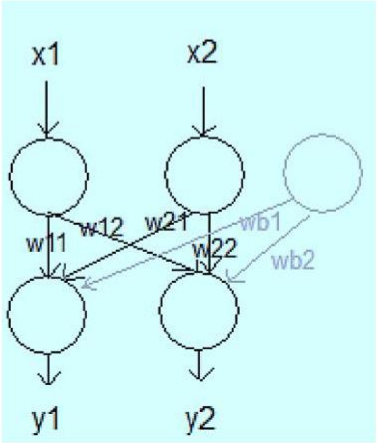
Non-linearly separable:
We can NOT use SLP

SLP example:

- Given the following Single Layer Perceptron (SLP) Artificial Neural Network (ANN), with 2 inputs x1 and x2 at the input layer, 2 outputs y1 and y2 at the output layer.
- Also, given the values presented in the table below for the inputs x1 and x2, target outputs t1 and t2, with initial weights $w_{11} = 0.1$, $w_{21} = 0.2$, $w_{12} = 0.2$, $w_{22} = 0.3$, $w_{b1} = -0.2$, $w_{b2} = 1$ and bias =1.

Show the details of training epochs considering usage of a step activation function with $\Theta = 0.1$

x1	x2	t1	t2
0	0	0	0
0	1	1	0
1	0	1	0
1	1	1	1



Answer

Epoch-1

x1	x2	w11	w21	wb1	w12	w22	wb2	y1	y2	t1	t2
0	0									0	0
0	1									1	0
1	0									1	0
1	1									1	1

Epoch-1

Initial weights

x1	x2	w11	w21	wb1	w12	w22	wb2	y1	y2	t1	t2
0	0	0.1	0.2	-0.2	0.2	0.3	1			0	0
0	1									1	0
1	0									1	0
1	1									1	1

Epoch-1

x1	x2	w11	w21	wb1	w12	w22	wb2	y1	y2	t1	t2
0	0	0.1	0.2	-0.2	0.2	0.3	1	0		0	0
0	1									1	0
1	0									1	0
1	1									1	1

Sum_y1 = (x1*w11)+(x2*w21)+(bias*wb1)
= (0 * 0.1) + (0 * 0.2) + (1* -0.2) = -0.2

-0.2 < Θ → y1=0

As Θ = 0.1 (threshold of activation function_y1)

Sum_y1>= Θ → y1=1
Sum_y1< Θ → y1=0

Epoch-1

x1	x2	w11	w21	wb1	w12	w22	wb2	y1	y2	t1	t2
0	0	0.1	0.2	-0.2	0.2	0.3	1	0	1	0	0
0	1									1	0
1	0									1	0
1	1									1	1

Sum_y2 = (x1*w12)+(x2*w22)+(bias*wb2)
= (0 * 0.2) + (0 * 0.3) + (1* 1) = 1

1 >= Θ → y2=1

As Θ = 0.1 (threshold of activation function_y2)

Sum_y2>= Θ → y2=1
Sum_y2< Θ → y2=0

Epoch-1

x1	x2	w11	w21	wb1	w12	w22	wb2	y1	y2	t1	t2
0	0	0.1	0.2	-0.2	0.2	0.3	1	0	1	0	0
0	1	0.1	0.2	-0.2						1	0
1	0									1	0
1	1									1	1

Error of y1 (e1) = target output (t1) – observed output (y1) = 0 – 0 = 0

As e1 = 0 → Then, No error for y1 → use the same weights for y1 in the next step

Epoch-1

x1	x2	w11	w21	wb1	w12	w22	wb2	y1	y2	t1	t2
0	0	0.1	0.2	-0.2	0.2	0.3	1	0	1	0	0
0	1	0.1	0.2	-0.2						1	0
1	0									1	0
1	1									1	1

Error of y2 (e2) = target output (t2) – observed output (y2) = 0 – 1 = -1
As e2 = -1 (≠ 0) → Then, there's an error in y2

→ use perceptron learning algorithm to update the weights of y2 for the next step

Epoch-1

x1	x2	w11	w21	wb1	w12	w22	wb2	y1	y2	t1	t2
0	0	0.1	0.2	-0.2	0.2	0.3	1	0	1	0	0
0	1	0.1	0.2	-0.2	0.2	0.3	0			1	0
1	0									1	0
1	1									1	1

→ $w12_{new} = w12_{old} + \Delta\, w12$; $\Delta\, w12 = x1 * e2$ → $w12_{new} = 0.2 + (0*-1) = \mathbf{0.2}$

→ $w22_{new} = w22_{old} + \Delta\, w22$; $\Delta\, w22 = x2 * e2$ → $w22_{new} = 0.3 + (0*-1) = \mathbf{0.3}$

→ $wb2_{new} = wb2_{old} + \Delta\, wb2$; $\Delta\, wb2 = bias * e2$ → $wb2_{new} = 1 + (1*-1) = \mathbf{0}$

Epoch-1

x1	x2	w11	w21	wb1	w12	w22	wb2	y1	y2	t1	t2
0	0	0.1	0.2	-0.2	0.2	0.3	1	0	1	0	0
0	1	0.1	0.2	-0.2	0.2	0.3	0	0		1	0
1	0									1	0
1	1									1	1

Sum_y1 = (x1*w11)+(x2*w21)+(bias*wb1)
= (0 * 0.1) + (1 * 0.2) + (1* -0.2) = 0

$0 < \Theta \rightarrow y1=0$

As Θ = 0.1 (threshold of activation function_y1)

Sum_y1>= $\Theta \rightarrow$ y1=1
Sum_y1< $\Theta \rightarrow$ y1=0

Epoch-1

x1	x2	w11	w21	wb1	w12	w22	wb2	y1	y2	t1	t2
0	0	0.1	0.2	-0.2	0.2	0.3	1	0	1	0	0
0	1	0.1	0.2	-0.2	0.2	0.3	0	0	1	1	0
1	0									1	0
1	1									1	1

Sum_y2 = (x1*w12)+(x2*w22)+(bias*wb2)
= (0 * 0.2) + (1 * 0.3) + (1* 0) = 0.3

0.3 >= Θ → y2=1

As Θ = 0.1 (threshold of activation function_y2)

Sum_y2>= Θ → y2=1
Sum_y2< Θ → y2=0

Epoch-1

x1	x2	w11	w21	wb1	w12	w22	wb2	y1	y2	t1	t2
0	0	0.1	0.2	-0.2	0.2	0.3	1	0	1	0	0
0	1	0.1	0.2	-0.2	0.2	0.3	0	0	1	1	0
1	0									1	0
1	1									1	1

Error of y1 (e1) = t1 - y1 = 1 – 0 = 1
Error of y2 (e2) = t2 – y2 = 0 - 1= -1

→ use perceptron learning algorithm to update the weights of y1 & y2 for the next step

Epoch-1

x1	x2	w11	w21	wb1	w12	w22	wb2	y1	y2	t1	t2
0	0	0.1	0.2	-0.2	0.2	0.3	1	0	1	0	0
0	1	0.1	0.2	-0.2	0.2	0.3	0	0	1	1	0
1	0	0.1	1.2	0.8	0.2	-0.7	-1			1	0
1	1									1	1

Updating weights of perceptron y1

→ $w11_{new} = w11_{old} + (x1 * e1) = 0.1 + (0*1) = \mathbf{0.1}$
→ $w21_{new} = w21_{old} + (x2 * e1) = 0.2 + (1*1) = \mathbf{1.2}$
→ $wb1_{new} = wb1_{old} + (bias * e1) = -0.2 + (1*1) = \mathbf{0.8}$

Updating weights of perceptron y2

→ $w12_{new} = w12_{old} + (x1 * e2) = 0.2 + (0*-1) = \mathbf{0.2}$
→ $w22_{new} = w22_{old} + (x2 * e2) = 0.3 + (1*-1) = \mathbf{-0.7}$
→ $wb2_{new} = wb2_{old} + (bias * e2) = 0 + (1*-1) = \mathbf{-1}$

Epoch-1

x1	x2	w11	w21	wb1	w12	w22	wb2	y1	y2	t1	t2
0	0	0.1	0.2	-0.2	0.2	0.3	1	0	1	0	0
0	1	0.1	0.2	-0.2	0.2	0.3	0	0	1	1	0
1	0	0.1	1.2	0.8	0.2	-0.7	-1	1	0	1	0
1	1									1	1

Sum_y1 = (x1*w11)+(x2*w21)+(bias*wb1)
= (1 * 0.1) + (0 * 1.2) + (1* 0.8) = 0.9

0.9 >= Θ → y1=1; e1= 1 – 1 = 0

Sum_y2 = (x1*w12)+(x2*w22)+(bias*wb2)
= (1 * 0.2) + (0 * -0.7) + (1* -1) = -0.8

-0.8 < Θ → y2=0; e2= 0 – 0 = 0

As e1 and e2 = 0 → Then, use the same weights for y1 and y2 in the next step

Epoch-1

x1	x2	w11	w21	wb1	w12	w22	wb2	y1	y2	t1	t2
0	0	0.1	0.2	-0.2	0.2	0.3	1	0	1	0	0
0	1	0.1	0.2	-0.2	0.2	0.3	0	0	1	1	0
1	0	0.1	1.2	0.8	0.2	-0.7	-1	1	0	1	0
1	1	0.1	1.2	0.8	0.2	-0.7	-1	1	0	1	1

Sum_y1 = (x1*w11)+(x2*w21)+(bias*wb1)
= (1 * 0.1) + (1 * 1.2) + (1* 0.8) = 2.1

→ 2.1 >= Θ → y1=1; e1= 1 – 1 = 0

Sum_y2 = (x1*w12)+(x2*w22)+(bias*wb2)
= (1 * 0.2) + (1 * -0.7) + (1* -1) = -1.5

→ -1.5 < Θ → y2=0; e2= 1 – 0 = 1

As e2 = 1 (≠ 0) → Then, there's an error in y2

→ use perceptron learning algorithm to update the weights of y2 for the next epoch

Epoch-1

x1	x2	w11	w21	wb1	w12	w22	wb2	y1	y2	t1	t2
0	0	0.1	0.2	-0.2	0.2	0.3	1	0	1	0	0
0	1	0.1	0.2	-0.2	0.2	0.3	0	0	1	1	0
1	0	0.1	1.2	0.8	0.2	-0.7	-1	1	0	1	0
1	1	0.1	1.2	0.8	0.2	-0.7	-1	1	0	1	1
		0.1	1.2	0.8	1.2	0.3	0				

Updating weights of perceptron y2

→ $w12_{new} = w12_{old} + (x1 * e2) = 0.2 + (1*1) = \mathbf{1.2}$

→ $w22_{new} = w22_{old} + (x2 * e2) = -0.7 + (1*1) = \mathbf{0.3}$

→ $wb2_{new} = wb2_{old} + (bias * e2) = -1 + (1*1) = \mathbf{0}$

Complete answer

Epoch-1 to 4

EPOCH-1																	
	x1	x2	w11	w21	wb1	w12	w22	wb2	bias	sum-y1	sum-y2	y1	y2	t1	t2	e1	e2
	0	0	0.1	0.2	-0.2	0.2	0.3	1	1	-0.2	1	0	1	0	0	0	-1
	0	1	0.1	0.2	-0.2	0.2	0.3	0	1	0	0.3	0	1	1	0	1	-1
	1	0	0.1	1.2	0.8	0.2	-0.7	-1	1	0.9	-0.8	1	0	1	0	0	0
	1	1	0.1	1.2	0.8	0.2	-0.7	-1	1	2.1	-1.5	1	0	1	1	0	1
	new weights to epoch2		0.1	1.2	0.8	1.2	0.3	0									
EPOCH-2																	
	x1	x2	w11	w21	wb1	w12	w22	wb2	bias	sum-y1	sum-y2	y1	y2	t1	t2	e1	e2
	0	0	0.1	1.2	0.8	1.2	0.3	0	1	0.8	0	1	0	0	0	-1	0
	0	1	0.1	1.2	-0.2	1.2	0.3	0	1	1	0.3	1	1	1	0	0	-1
	1	0	0.1	1.2	-0.2	1.2	-0.7	-1	1	-0.1	0.2	0	1	1	0	1	-1
	1	1	1.1	1.2	0.8	0.2	-0.7	-2	1	3.1	-2.5	1	0	1	1	0	1
	new weights to epoch3		1.1	1.2	0.8	1.2	0.3	-1									
EPOCH-3																	
	x1	x2	w11	w21	wb1	w12	w22	wb2	bias	sum-y1	sum-y2	y1	y2	t1	t2	e1	e2
	0	0	1.1	1.2	0.8	1.2	0.3	-1	1	0.8	-1	1	0	0	0	-1	0
	0	1	1.1	1.2	-0.2	1.2	0.3	-1	1	1	-0.7	1	0	1	0	0	0
	1	0	1.1	1.2	-0.2	1.2	0.3	-1	1	0.9	0.2	1	1	1	0	0	-1
	1	1	1.1	1.2	-0.2	0.2	0.3	-2	1	2.1	-1.5	1	0	1	1	0	1
	new weights to epoch4		1.1	1.2	-0.2	1.2	1.3	-1									
EPOCH-4																	
	x1	x2	w11	w21	wb1	w12	w22	wb2	bias	sum-y1	sum-y2	y1	y2	t1	t2	e1	e2
	0	0	1.1	1.2	-0.2	1.2	1.3	-1	1	-0.2	-1	0	0	0	0	0	0
	0	1	1.1	1.2	-0.2	1.2	1.3	-1	1	1	0.3	1	1	1	0	0	-1
	1	0	1.1	1.2	-0.2	1.2	0.3	-2	1	0.9	-0.8	1	0	1	0	0	0
	1	1	1.1	1.2	-0.2	1.2	0.3	-2	1	2.1	-0.5	1	0	1	1	0	1
	new weights to epoch5		1.1	1.2	-0.2	2.2	1.3	-1									

Perceptron (y1) has completed training after epoch-4

Epoch-5 to 8

EPOCH-5																	
	x1	x2	w11	w21	wb1	w12	w22	wb2	bias	sum-y1	sum-y2	y1	y2	t1	t2	e1	e2
	0	0	1.1	1.2	-0.2	2.2	1.3	-1	1	-0.2	-1	0	0	0	0	0	0
	0	1	1.1	1.2	-0.2	2.2	1.3	-1	1	1	0.3	1	1	1	0	0	-1
	1	0	1.1	1.2	-0.2	2.2	0.3	-2	1	0.9	0.2	1	1	1	0	0	-1
	1	1	1.1	1.2	-0.2	1.2	0.3	-3	1	2.1	-1.5	1	0	1	1	0	1
	new weights to epoch6		1.1	1.2	-0.2	2.2	1.3	-2									
EPOCH6																	
	x1	x2	w11	w21	wb1	w12	w22	wb2	bias	sum-y1	sum-y2	y1	y2	t1	t2	e1	e2
	0	0	1.1	1.2	-0.2	2.2	1.3	-2	1	-0.2	-2	0	0	0	0	0	0
	0	1	1.1	1.2	-0.2	2.2	1.3	-2	1	1	-0.7	1	0	1	0	0	0
	1	0	1.1	1.2	-0.2	2.2	1.3	-2	1	0.9	0.2	1	1	1	0	0	-1
	1	1	1.1	1.2	-0.2	1.2	1.3	-3	1	2.1	-0.5	1	0	1	1	0	1
	new weights to epoch7		1.1	1.2	-0.2	2.2	2.3	-2									
EPOCH-7																	
	x1	x2	w11	w21	wb1	w12	w22	wb2	bias	sum-y1	sum-y2	y1	y2	t1	t2	e1	e2
	0	0	1.1	1.2	-0.2	2.2	2.3	-2	1	-0.2	-2	0	0	0	0	0	0
	0	1	1.1	1.2	-0.2	2.2	2.3	-2	1	1	0.3	1	1	1	0	0	-1
	1	0	1.1	1.2	-0.2	2.2	1.3	-3	1	0.9	-0.8	1	0	1	0	0	0
	1	1	1.1	1.2	-0.2	2.2	1.3	-3	1	2.1	0.5	1	1	1	1	0	0
	new weights to epoch8		1.1	1.2	-0.2	2.2	1.3	-3									
EPOCH-8																	
	x1	x2	w11	w21	wb1	w12	w22	wb2	bias	sum-y1	sum-y2	y1	y2	t1	t2	e1	e2
	0	0	1.1	1.2	-0.2	2.2	1.3	-3	1	-0.2	-3	0	0	0	0	0	0
	0	1	1.1	1.2	-0.2	2.2	1.3	-3	1	1	-1.7	1	0	1	0	0	0
	1	0	1.1	1.2	-0.2	2.2	1.3	-3	1	0.9	-0.8	1	0	1	0	0	0
	1	1	1.1	1.2	-0.2	2.2	1.3	-3	1	2.1	0.5	1	1	1	1	0	0
	new weights to epoch9		1.1	1.2	-0.2	2.2	1.3	-3									

Optimal weights for the SLP NN

No errors → Stop training

Both perceptron (y1 & y2) have completed training after epoch-8